Chapter 1
The Machine That Thinks

In which we build the smallest possible CPU and discover that every computer, no matter how sophisticated, is really just one very fast, very simple loop.

STAGE 1 · BUILD A TINY CPU SIMULATOR — LANGUAGE: PYTHON

1.0 What Does a CPU Actually Do?

Before writing any code, it's worth sitting with a question that sounds too simple to be useful: **what does a CPU actually do?**

Not what it *is*. Not what it's made of. What does it *do*, moment to moment?

> **KEY INSIGHT**
>
> A CPU reads a number from memory, figures out what that number means, does what it says, then reads the next number. That's it. Over and over, billions of times per second.

Everything — your browser, your operating system, every program you have ever run — is ultimately that loop. A machine fetching numbers, interpreting them as instructions, and acting on them.

This is called the **fetch–decode–execute cycle**, and it is the heartbeat of every computer ever built.

When we build a virtual machine, we are building a program that *pretends to be that machine*. To pretend convincingly, we first have to understand precisely what we are pretending to be. So we start here, at the smallest possible scale.

1.1 The Registers

A CPU needs a small number of very fast storage slots to work with. These are called **registers** — think of them as the CPU's scratchpad, a handful of named variables the CPU can read and write instantly, without going out to memory.

Real CPUs have many registers with specialized purposes. For now, we only need two categories:

General-purpose registers hold data the CPU is actively working on. We'll call ours `R0` through `R3`. Four is enough to start.

The Program Counter (PC) is the most important register in the entire machine. It holds the *address of the next instruction to execute*. Every time the CPU fetches an instruction, it reads from the address currently in `PC`, then advances `PC` forward.

The PC is the cursor of the machine — it tells the CPU where it is in the program at every instant.

1.2 Memory as a Simple Array

Memory is just a big array of numbers. Each slot has an address (its index) and holds a value. Instructions live in memory. Data lives in memory. The CPU reads instructions from memory using the PC, and reads or writes data to memory using addresses.

For our simulator, memory will literally be a Python list.

1.3 Instructions Are Just Numbers

Here is the key insight of this chapter: **instructions are just numbers**. A CPU does not know the difference between an instruction and data at the level of raw storage — the interpretation comes entirely from *how the CPU treats it*.

When the CPU reads a value from the address held in `PC`, it treats that value as an instruction. The **decode** step is what figures out which operation that number represents.

We will design a tiny, clean instruction set. Each instruction is a simple integer — an **opcode** — and we get to define what each opcode means.

OPCODE	NAME	MEANING
`0`	HALT	Stop execution
`1`	LOAD_IMM	Load an immediate value into a register
`2`	ADD	Add two registers, store the result
`3`	PRINT_REG	Print the value of a register

"Immediate" means the value is *embedded directly in the instruction stream* — the number to load follows immediately after the opcode in memory.

1.4 Your First CPU

Below is the smallest valid version of our CPU simulator. Every line matters — read it carefully rather than skimming.

CPU.PY

```
def main():
    memory = [0] * 256          # flat memory array
    registers = [0, 0, 0, 0]  # R0..R3
    pc = 0                       # program counter

    # LOAD_IMM reg,val => [1,reg,val] | ADD dst,s1,s2 => [2,dst,s1,s2]
    # PRINT_REG reg    => [3,reg]     | HALT          => [0]
    program = [
        1, 0, 7,
        1, 1, 5,
        2, 2, 0, 1,
        3, 2,
        0,
    ]

    for i, word in enumerate(program):
        memory[i] = word

    running = True

    while running:
        opcode = memory[pc]  # FETCH

        if opcode == 0:  # HALT
            print("[CPU] HALT — execution stopped.")
            running = False

        elif opcode == 1:  # LOAD_IMM reg, value
            reg = memory[pc + 1]
```

```
            val = memory[pc + 2]
            registers[reg] = val
            pc += 3

        elif opcode == 2:  # ADD dst, src1, src2
            dst  = memory[pc + 1]
            src1 = memory[pc + 2]
            src2 = memory[pc + 3]
            registers[dst] = registers[src1] + registers[src2]
            pc += 4

        elif opcode == 3:  # PRINT_REG reg
            reg = memory[pc + 1]
            print(f"[CPU] R{reg} = {registers[reg]}")
            pc += 2

        else:
            print(f"[CPU] ERROR — unknown opcode {opcode} at PC={pc}")
            running = False

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

1.5 Run It

Save this as `cpu.py` and run it:

```
python cpu.py
```

You should see:

```
[CPU] R2 = 12
[CPU] HALT — execution stopped.
```

12 = 7 + 5. Your CPU computed that.

1.6 What Just Happened

Trace through what the machine did, step by step:

1. `PC = 0`. Fetch opcode `1` → `LOAD_IMM`. Read `reg=0`, `val=7`. Set `R0 = 7`. Advance `PC` to `3`.
2. `PC = 3`. Fetch opcode `1` → `LOAD_IMM`. Read `reg=1`, `val=5`. Set `R1 = 5`. Advance `PC` to `6`.
3. `PC = 6`. Fetch opcode `2` → `ADD`. Read `dst=2`, `src1=0`, `src2=1`. Set `R2 = R0 + R1 = 12`. Advance `PC` to `10`.
4. `PC = 10`. Fetch opcode `3` → `PRINT_REG`. Read `reg=2`. Print `R2 = 12`. Advance `PC` to `12`.
5. `PC = 12`. Fetch opcode `0` → `HALT`. Stop.

The PC marched through memory. Every opcode told it how far to jump forward. The CPU had no idea it was computing 7+5 — it just fetched numbers and followed rules.

> **THIS IS HOW EVERY CPU WORKS**
>
> Yours. Mine. The one in your phone. The one that will eventually run Linux inside the VM we build together in this apprenticeship.

CHAPTER 1 — REVIEW QUESTIONS

1. What is the Program Counter, and what happens to it when the CPU executes a `LOAD_IMM` instruction?
2. Why does `ADD` advance `PC` by 4, but `HALT` doesn't advance `PC` at all?
3. If you changed `program[2]` from `7` to `42`, what would the output be? Why?
4. What would happen if the PC contained an address where no instruction existed — just a `0` value in memory?

CHAPTER 1 — IMPLEMENTATION TASK

Extend the CPU with two new instructions:

OPCODE	NAME	FORMAT	MEANING
`4`	SUB	`[4, dst, src1, src2]`	`dst = src1 - src2`
`5`	MOV	`[5, dst, src]`	Copy register: `dst = src`

Then write a program that:

- Loads `10` into `R0`
- Loads `3` into `R1`
- Subtracts `R1` from `R0` into `R2`
- Copies `R2` into `R3`
- Prints both `R2` and `R3`
- Halts

Expected output:

```
[CPU] R2 = 7
[CPU] R3 = 7
[CPU] HALT — execution stopped.
```

---

*Next: **Chapter 2 — Branching and the Jump Instruction**, where the CPU gains the ability to make decisions, and you'll see for the first time how loops and conditionals are actually implemented in hardware.*